Joint Service Caching, Task Offloading, and Service Delivery in Mobile Edge Computing for Latency Critical UAV Networks

Dipankar Ch Barman, Sayantan Chakraborty, Abhishek Hazra, *Member, IEEE*, and Nabajyoti Mazumdar, *Senior Member, IEEE*

***Abstract*—In modern mobile edge computing (MEC) systems, effectively integrating service caching, task offloading, and service delivery is crucial for ensuring high network quality and enhanced user experience. In this article, we introduce a novel joint optimization framework that employs Uncrewed aerial vehicles (UAVs) as dynamic edge computing nodes, collaborating with service collection units (SCUs) that aggregate service requests from Internet of Things (IoT) devices. In the proposed system, UAVs equipped with small-capacity edge servers can process computational tasks locally or forward them to a base station (BS) via the backhaul when necessary. Both user layers (UEs) and UAVs dynamically cache programs using an adaptive algorithm based on task frequency and program weight, which maximizes cache hit rates and expedites service execution. A joint optimization problem is formulated to minimize the overall end-to-end latency through coordinated task offloading, CPU allocation, caching, and UAV scheduling, subject to energy, storage, and mobility constraints. Our resource-aware task offloading and UAV trajectory planning further enable efficient service responsiveness and minimize reliance on the backhaul link. The overall objective is to reduce maximum task latency and backhaul congestion across the network. Extensive simulation results show that our framework significantly outperforms standard baselines, including random caching, most caching, and calculation delay and energy consumption multiobjective optimization problem (CDECMOP), achieving up to 18.5% lower energy consumption and 23.6% lower latency across diverse network scenarios. These findings underscore the effectiveness of the proposed UE-UAV cooperative caching and computation strategy for scalable, energy-efficient, and low-latency UAV-assisted MEC systems.**

***Index Terms*—Mobile edge computing (MEC), service caching, service collection units (SCUs), service delivery, task offloading, uncrewed aerial vehicles (UAVs).**

Received 4 July 2025; revised 10 November 2025; accepted 3 December 2025. Date of publication 14 January 2026; date of current version 2 June 2026. This work was supported by the Anusandhan National Research Foundation (ANRF), Government of India, under Grant EEQ/2022/000400. *(Corresponding author: Nabajyoti Mazumdar.)*

Dipankar Ch Barman, Sayantan Chakraborty, and Nabajyoti Mazumdar are with the Department of Information Technology, Indian Institute of Information Technology, Allahabad 210015, India (e-mail: rsi2023004@iiita.ac.in; mml2022007@iiita.ac.in; nabajyoti@iiita.ac.in).

Abhishek Hazra is with the Department of Computer Science and Engineering, Indian Institute of Information Technology, Sri City 517646, India (e-mail: abhishek.hazra1@gmail.com).

Digital Object Identifier 10.1109/TCSS.2025.3643164

I. Introduction

In today's era of intelligent connectivity, the rapid expansion of mobile devices and data-intensive applications has led to a growing demand for low-latency, high-throughput, and energy-efficient communication infrastructures [1], [2]. MEC has emerged as a key paradigm to address these challenges by offloading computationally intensive tasks from resource-limited user layer (UE) to nearby edge servers, thereby improving service response and reducing core network congestion [3], [4]. Uncrewed aerial vehicles (UAVs) have become an essential enabler of next-generation MEC systems due to their mobility, flexibility, and ability to provide on-demand edge services in areas lacking terrestrial infrastructure [5], [6]. UAV-assisted MEC enables efficient computation offloading and adaptive coverage in dynamic and delay-sensitive environments, such as industrial automation, emergency response, and large-scale Internet of Things (IoT) deployments [7]. However, optimizing the allocation of limited communication, computation, and caching resources in such systems remains a major challenge. Efficient coordination among these components is crucial to reduce latency, balance energy consumption, and maintain quality of service (QoS) [8], [9], [10].

The task assignment problem in UAV-assisted MEC involves determining how computational tasks should be distributed among available edge resources while considering UAV mobility, link quality, and energy constraints [11], [12]. At the same time, UAV trajectory planning plays a critical role in ensuring coverage and minimizing travel distance, flight energy, and service delay [7], [13]. Most existing studies have focused on isolated subproblems such as task offloading and trajectory optimization, without fully capturing their inherent interdependencies. Furthermore, the limited caching capacity at UAVs and user devices poses additional challenges for reducing backhaul usage and improving data availability. Consequently, there is a strong need for an integrated framework that jointly addresses service caching, task offloading, and trajectory planning to enhance end-to-end system performance. In real-world UAV-assisted MEC scenarios, effective and timely task execution is vital for applications such as disaster management, real-time surveillance, and vehicular coordination, where computational

2329-924X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

results must be delivered within strict deadlines [14], [15], [16], [17]. The existing literature has primarily concentrated on optimizing computation offloading or path planning but has paid limited attention to the equally critical challenge of service delivery and cache management [18]. Integrating caching mechanisms into the offloading and delivery pipeline can significantly reduce latency and improve data availability by retaining popular services closer to end users. Therefore, it is essential to develop a holistic strategy that integrates these interdependent components within a unified framework for optimization.

A. Motivation

The motivation behind this article stems from the need to develop an approach that overcomes intrinsic shortcomings and provides a comprehensive solution to the aforementioned challenges. Existing works often overlook task response delivery mechanisms, which are crucial for ensuring timely and reliable communication in networked systems. Most existing solutions address individual challenges, such as reducing latency and alleviating backhaul traffic; however, the real concern lies in achieving a joint solution, since these problems are inherently interconnected. This limitation reveals a research gap where further investigation is required. Implementing caching to retain services or content in close proximity to users is essential to mitigate delay issues. Such caching ensures that services and data are readily available near the end users, resulting in faster response times. Therefore, this research aims to fill the identified gap by proposing a holistic framework that integrates both caching and robust task response delivery mechanisms. The proposed approach effectively addresses the above-mentioned issues and enhances overall system utilization, responsiveness, and robustness.

B. Our Contribution

Building on these motivations, this article proposes an integrated optimization framework that unifies service caching, task offloading, and service delivery in UAV-assisted MEC networks. The main contributions are summarized as follows.

1) We propose a unified framework that jointly optimizes service caching, task offloading, and service delivery with the primary goal of minimizing end-to-end latency. The design inherently enhances energy efficiency and resource utilization through improved caching and computation locality.
2) A dynamic caching mechanism is developed that combines task request frequency and program weight factors for adaptive cache updates at both UEs and UAVs, significantly improving cache hit rates and response efficiency.
3) We design a resource-aware offloading and UAV trajectory optimization strategy to balance computational load, communication cost, and service delivery under mobility and capacity constraints.
4) By prioritizing local and UAV-level processing, the proposed approach reduces backhaul congestion and enhances scalability. Simulation results validate the effectiveness of the framework, demonstrating substantial latency reduction and near-optimal performance compared with benchmark methods.

The remainder of this article is organized as follows. Section II discusses the related work. Section III presents the system model and problem formulation. Section IV explains the theoretical analysis, followed by the proposed methodology in Section V. Section VI provides simulation results and performance evaluation, and Section VII concludes the article.

II. Related Work

Recent studies on MEC have focused on improving computational efficiency, latency, and energy utilization through task offloading, caching, and resource allocation techniques [4], [19], [20]. With the emergence of UAVs as mobile edge servers, UAV-assisted MEC has become a prominent research direction, offering flexibility and extended coverage compared to terrestrial-only systems [21]. Existing studies on UAV-assisted MEC can be broadly categorized into three analytical paradigms: 1) computation content caching; 2) computation service caching; and 3) service caching with UAVs. Each emphasizes different trade-offs between latency, resource utilization, and adaptability.

A. Computation Content Caching

Early studies focused on caching task-related data, aiming to reduce transmission delay and dependence on backhaul. Techniques such as periodic caching decisions [22], coded caching for nonterrestrial IoT [23], and multimedia data storage [24] improved delivery speed but remained data-centric. Learning-based approaches [25], [26] introduced adaptivity by predicting content popularity, yet they remained limited by storage constraints and scalability when data volumes increased. These approaches effectively address the delay caused by large data transmission but overlook the fact that caching raw content is inefficient for computation-intensive MEC tasks. Since edge devices often require executable programs rather than static data, this paradigm struggles to support real-time computation reuse.

B. Computation Service Caching

To overcome data-centric limitations, later research explored caching executable services or models instead of raw content [27]. By storing program libraries locally, service caching minimizes reloading latency and supports diverse computation tasks. Works such as [28] and [29] demonstrated that joint optimization of service caching and offloading decisions can significantly reduce latency. Although this paradigm improves task responsiveness and reduces backhaul usage, most existing models treat caching and offloading as loosely coupled problems. The interdependence between caching placement, resource allocation, and UAV trajectory remains underexplored, limiting the adaptability of these methods in dynamic MEC environments.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

C. Service Caching With UAVs

The most recent trend integrates UAV mobility into caching and offloading decisions, making resource management spatially and temporally adaptive. Multiagent learning [30], stochastic scheduling [31], and deep reinforcement learning (DRL)-based joint optimization [32] have demonstrated improved decision-making efficiency. Optimization frameworks based on Lyapunov control [5] and multitask offloading in satellite IoT scenarios [33] extend these capabilities to hybrid networks. While UAV-assisted caching introduces dynamic flexibility, these works typically optimize subcomponents, either caching, offloading, or trajectory, individually. None explicitly addresses how these dimensions interact to jointly minimize latency and backhaul load. Moreover, the task-response delivery stage, critical for end-to-end latency, is often omitted from performance formulations.

From the above analysis, it is evident that existing approaches often optimize caching or offloading in isolation, leading to inefficiencies in overall system coordination. Caching decisions influence offloading strategies, and UAV trajectory planning affects both communication efficiency and service delivery. Hence, a unified and cross-layer optimization framework is required to jointly address caching, offloading, and service delivery while minimizing latency and backhaul congestion. This article aims to bridge these gaps by developing an integrated UAV-assisted MEC framework that simultaneously optimizes cache placement, task offloading, and UAV trajectory planning. The proposed design leverages adaptive caching to handle variations in task demand and introduces a service delivery strategy that ensures efficient result transmission and reduced end-to-end latency. By jointly addressing these interrelated components, the framework enhances scalability, energy efficiency, and responsiveness in dynamic MEC environments.

III. System Model and Problem Formulation

This section introduces principal components, interactions, and operations, which form the base model of our system. The components of our model are shown in Fig. 1 and notations are summarized in Table I.

A. Network Model

The proposed UAV-assisted MEC system operates within a three-tier hierarchical framework consisting of the UEs, the intermediate service aggregation layer (SCUs), and the aerial edge layer (UAVs). A central base station (BS) serves as the backhaul node, providing additional computing and storage support. Each UE generates computation tasks and transmits them to its associated SCU, which functions as a ground-level aggregation and relay node. The UAVs perform computation at the edge, while the BS handles cloud-level processing and program updates for the UAVs. All entities are deployed within a service area of size $A \times A$ m^2, where UEs $U = \{\mathrm{ue}_1, \ldots, \mathrm{ue}_{N_{\mathrm{UE}}}\}$ and SCUs $S = \{s_1, \ldots, s_{N_{\mathrm{SCU}}}\}$ are located on the ground, while UAVs $V = \{\mathrm{uv}_1, \ldots, \mathrm{uv}_{N_{\mathrm{UAV}}}\}$ operate at a fixed altitude H, satisfying $N_{\mathrm{UE}} > N_{\mathrm{SCU}} > N_{\mathrm{UAV}}$. Each entity's position is denoted as $q_{\mathrm{ue}_j} = (x_{\mathrm{ue}_j}, y_{\mathrm{ue}_j})$, $q_{s_\ell} = (x_{s_\ell}, y_{s_\ell})$, $q_{\mathrm{uv}_m} = (x_{\mathrm{uv}_m}, y_{\mathrm{uv}_m})$, and $q_{\mathrm{BS}} = (x_{\mathrm{BS}}, y_{\mathrm{BS}})$. UAVs are equipped with directional antennas of beamwidth θ and maintain altitude H, adjusting $(x_{\mathrm{uv}_m}, y_{\mathrm{uv}_m})$ for trajectory optimization. UE-SCU and SCU-UAV links are formed based

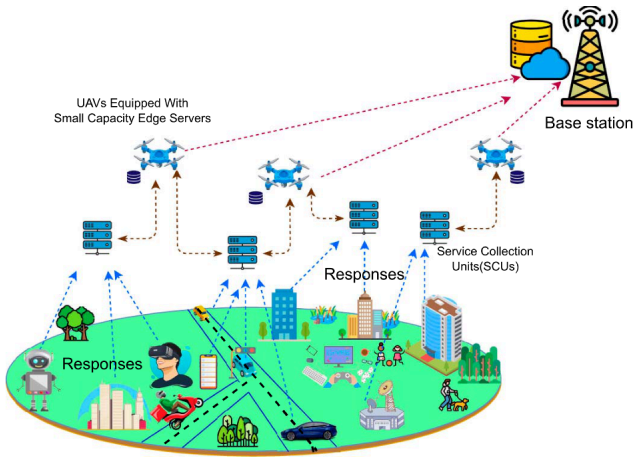


Fig. 1. Illustration of network model.

TABLE I
List of Notations

Symbol	Description
A	Side length of the square service area (m).
H	Fixed altitude of UAVs (m).
$N_{\mathrm{UE}}, N_{\mathrm{SCU}}, N_{\mathrm{UAV}}$	Number of UEs, SCUs, and UAVs, respectively.
$M^{\max}_{\mathrm{tasks},\mathrm{uv}_m}$	Maximum number of concurrent tasks UAV uv_m can handle.
$C^{\max}_{\mathrm{uv}_m}, C^{\max}_{\mathrm{ue}_j}$	Maximum caching capacity of UAV uv_m and UE ue_j.
$E^{\max}_{\mathrm{uv}_m}, E^{\max}_{\mathrm{ue}_j}$	Maximum energy budget of UAV uv_m and UE ue_j.
$D^{\max}_i$	Maximum completion deadline for task t_i.
$\varphi^{\mathrm{cycle}}_i$	Number of CPU cycles required to process task t_i.
δ_i	Input data size of task t_i (bits).
l_j	Memory size of program P_j.
$d_{a,b}$	Euclidean distance between node a and b.
$B_{a,b}$	Bandwidth allocated to link (a, b).
$R_{a,b}$	Achievable data rate between nodes a and b.
P^{tx}_a	Transmit power of node a.
γ_n	Effective CPU capacitance coefficient of node n.
$\mathcal{F}(C^{\mathrm{cache}}, G, H, Q)$	Objective function representing task latency based on caching, offloading, CPU allocation, and UAV trajectory.
C^{cache}	Cache decision matrix; $C^{\mathrm{cache}}_{n,P_i} = 1$ if program P_i is stored at node n.
G	Offloading matrix; $g_{\mathrm{ue}_j,n} = 1$ if task t_i of UE ue_j executes at node n.
H	CPU allocation vector; $h_{\mathrm{ue}_j,i}$ is the CPU fraction assigned to task t_i.
Q	UAV trajectory set; $(x_{\mathrm{uv}_m}, y_{\mathrm{uv}_m})$ denotes UAV uv_m 's coordinates.
$q_{\mathrm{ue}_j}, q_{s_\ell}, q_{\mathrm{uv}_m}, q_{\mathrm{BS}}$	Spatial coordinates of UE ue_j, SCU s_ℓ, UAV uv_m, and BS.
$g_{\mathrm{ue}_j,n} \in \{0, 1\}$	Offloading indicator: 1 if UE ue_j offloads task to node n.
$h_{n,i}$	CPU resource fraction allocated by node n to task t_i.
$f_{n,i}$	CPU frequency assigned to task t_i at node n.
$C^{\mathrm{cache}}_{n,P_i} \in \{0, 1\}$	Cache indicator: 1 if program P_i is stored at node n.
$T^{\mathrm{exe}}_n(t_i)$	Execution time of task t_i at node n.
$T^{\mathrm{comm}}(t_i)$	Communication delay for task t_i.
$\mathbb{L}(t_i)$	End-to-end latency of task t_i.
$E^{\mathrm{comp}}_{n,i}, E^{\mathrm{comm}}_{n,i}$	Computation and communication energy components.
$E^{tx}_{\mathrm{ue}_j,s_\ell}, E^{\mathrm{fly}}_{\mathrm{uv}_m}$	Transmission and propulsion energy components.
$v_{\mathrm{uv}_m}, v^{tip}_{\mathrm{uv}_m}$	Flight and rotor-tip speeds of UAV uv_m.
$(x_{\mathrm{uv}_m}, y_{\mathrm{uv}_m})$	UAV's horizontal trajectory coordinates.
$T_{\mathrm{uv}_m}, T_{\mathrm{ue}_j}$	Task sets assigned to UAV uv_m and UE ue_j, respectively.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

on minimum Euclidean distance, ensuring association within antenna coverage. Each UAV can process up to $M^{\max}_{\text{tasks},\text{uv}_m}$ concurrent tasks and satisfies $C^{\max}_{\text{uv}_m} > C^{\max}_{\text{ue}_j}$ for caching capacity. Each UE generates tasks $T = \{t_1, t_2, \ldots, t_{N_{\text{tasks}}}\}$, where each task $t_i = (\delta_i, P_i, \varphi_i^{\text{cycle}}, D_i^{\max})$ includes its data size δ_i, required program P_i, required CPU cycle φ_i^{cycle}, and deadline $D_i^{\max}$. Each program P_i belongs to the global program library $P = \{p_1, p_2, \ldots, p_{N_{\text{progs}}}\}$, shared among the BS and UAVs. This hierarchy ensures that computation and communication occur across adjacent layers, UEs communicate with SCUs, SCUs communicate with UAVs, and UAVs communicate with the BS.

B. Communication Model

The UAV-assisted MEC system adopts a hierarchical communication structure linking UEs, SCUs, UAVs, and the BS. During the uplink (task offloading), data flows as $\text{UE} \to \text{SCU} \to \text{UAV} \to \text{BS}$, while in the downlink (result delivery), the flow reverses as $\text{BS} \to \text{UAV} \to \text{SCU} \to \text{UE}$. This bidirectional hierarchy ensures efficient task transmission and minimal backhaul dependence. Each UE ue_j first offloads its task to the nearest SCU s_ℓ, determined by

$$d_{\text{ue}_j, s_\ell} = \min_{s_\ell \in S} \|q_{\text{ue}_j} - q_{s_\ell}\|_2. \tag{1}$$

The SCU aggregates tasks and forward them to the nearest UAV uv_m satisfying cache and capacity constraints

$$d_{s_\ell, \text{uv}_m} = \min_{\text{uv}_m \in V} \|q_{s_\ell} - q_{\text{uv}_m}\|_2 \tag{2}$$

$$\text{subject to } M_{\text{tasks},\text{uv}_m} < M^{\max}_{\text{tasks},\text{uv}_m}. \tag{3}$$

If no UAV possesses the required program P_i or has available capacity, the SCU requests support from the BS via a wireless backhaul link. The BS transmits the missing program or computation results to the corresponding UAV, which then returns the processed output to the nearest SCU and subsequently to the UE. The achievable data rate between any two entities a and b, where $a, b \in \{\text{UE}, \text{SCU}, \text{UAV}, \text{BS}\}$, is given by

$$R_{a,b} = B_{a,b} \log_2 \left(1 + \frac{P_a^{\text{tx}} G_{a,b}}{N_0}\right) \tag{4}$$

where $B_{a,b}$ is the bandwidth allocated to the link (a, b), P_a^{tx} is the transmit power of node a, $G_{a,b}$ is the channel gain, and N_0 is the noise power density. The transmission delay for data of size δ_i from node a to node b is expressed as

$$T^{\text{tx}}_{a,b}(t_i) = \frac{\delta_i}{R_{a,b}}. \tag{5}$$

Hence, the total communication delay for a task t_i is the sum of its uplink and downlink delays, given as

$$T^{\text{up}}(t_i) = T^{\text{tx}}_{\text{UE,SCU}}(t_i) + T^{\text{tx}}_{\text{SCU,UAV}}(t_i) + T^{\text{tx}}_{\text{UAV,BS}}(p_r) \tag{6}$$

$$T^{\text{down}}(t_i) = T^{\text{tx}}_{\text{UAV,SCU}}(t_i) + T^{\text{tx}}_{\text{SCU,UE}}(t_i) \tag{7}$$

$$T^{\text{comm}}(t_i) = T^{\text{up}}(t_i) + T^{\text{down}}(t_i) \tag{8}$$

where $T^{\text{tx}}_{\text{UAV,BS}}(p_r)$ applies only if the UAV fetches program p_r from the BS.

C. Computational Model

The computational model describes how computation tasks are executed across UEs, UAVs, and the BS [16]. Depending on program availability, channel quality, and node capacity, a task may be executed locally or offloaded via its associated SCU to a UAV. The offloading decision of each UE ue_j is represented by a binary variable $g_{\text{ue}_j,n}$, where $n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\}$ denotes the selected execution node. A task is assigned to exactly one node, i.e., $\sum_{n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\}} g_{\text{ue}_j,n} = 1$. A node can execute task t_i only if the required program P_i is available in its cache, such that $g_{\text{ue}_j,\text{uv}_m} \le C^{\text{cache}}_{n,P_i}$ and $g_{\text{ue}_j,\text{ue}_j} \le C^{\text{cache}}_{n,P_i}$, where C^{cache}_{n,P_i} denotes the cache status (1 if available, 0 otherwise) of program P_i at node $n \in \{\text{ue}_j, \text{uv}_m\}$. Each node n operates with a maximum CPU frequency $F_n^{\max}$ that is distributed among its active tasks. If $h_{n,i}$ denotes the CPU fraction allocated to task t_i, the effective execution frequency and time are given by $f_{n,i} = h_{n,i} F_n^{\max}$ and

$$T_n^{\text{exe}}(t_i) = \frac{\varphi_i^{\text{cycle}}}{h_{n,i} F_n^{\max}} \tag{9}$$

where φ_i^{cycle} is the number of CPU cycles required for t_i. To ensure feasible computation scheduling, the cumulative CPU allocation across all tasks on any node must not exceed unity

$$\sum_{t_i \in T} h_{n,i} \le 1, \quad \forall n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\}. \tag{10}$$

Based on the offloading decision $g_{\text{ue}_j,n}$, the execution time is as follows:

$$T^{\text{exe}}(t_i) = \begin{cases} T^{\text{exe}}_{\text{ue}_j}(t_i), & \text{if executed locally at UE} \\ T^{\text{exe}}_{\text{uv}_m}(t_i), & \text{if offloaded to UAV} \\ T^{\text{exe}}_{\text{BS}}(t_i), & \text{if executed at BS.} \end{cases} \tag{11}$$

The total delay for task t_i comprises communication and computation latency, formulated as

$$\mathbb{L}(t_i) = T^{\text{comm}}(t_i) + T^{\text{exe}}(t_i) + T^{\text{deliv}}(t_i) \tag{12}$$

where $T^{\text{comm}}(t_i)$ includes the UE-SCU-UAV transmission time and $T^{\text{deliv}}(t_i)$ represents the delivery delay after processing. The latter depends on the UAV's current route and the number of SCUs visited before returning the processed result, expressed as

$$T^{\text{deliv}}(t_i) = N_{\text{SCU,Visited}} \Delta_{\text{hop}} \tag{13}$$

where $N_{\text{SCU,Visited}}$ denotes the number of SCUs the UAV has already served before reaching the target SCU, and Δ_{hop} is the average per-hop communication or hovering delay. The task completion must satisfy the deadline constraint $\mathbb{L}(t_i) \le D_i^{\max}, \ \forall t_i \in T$.

D. Caching Model

In the proposed framework, both UEs and UAVs are equipped with limited local caches that store frequently requested programs to reduce task transmission delays and minimize backhaul dependency. The caching capacity of each node is finite

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

and defined by its maximum available storage $C_n^{\max}$. Hence, the caching constraint for UEs and UAVs can be expressed as

$$\sum_{P_j \in P} C_{n,P_j}^{\text{cache}} l_j \leq C_n^{\max}, \quad \forall n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\} \tag{14}$$

where l_j denotes the storage size of program P_j. This inequality ensures that the total memory consumed by cached programs does not exceed the available storage capacity of either a UE or a UAV. The caching mechanism directly interacts with the offloading process, as a task t_i can be executed at a node only if its required program is locally available, satisfying the constraint $g_{\text{ue}_j,n} \leq C_{n,P_j}^{\text{cache}}$.

E. Energy Consumption Model

The total energy consumption in the proposed UAV-assisted MEC system arises from three sources: 1) local computation and transmission at UEs; 2) computation and communication at UAVs; and 3) propulsion energy during UAV flight. The overall system energy for all entities can be expressed as

$$E^{\text{total}} = \sum_{\text{ue}_j \in U} \left(E_{\text{ue}_j}^{\text{comp}} + E_{\text{ue}_j,s_\ell}^{\text{tx}} \right) + \sum_{\text{uv}_m \in V} \left(E_{\text{uv}_m}^{\text{comp}} + E_{\text{uv}_m}^{\text{comm}} + E_{\text{uv}_m}^{\text{fly}} \right). \tag{15}$$

1) UE Energy Consumption: When UE ue_j executes a task t_i locally, its computation energy follows the DVFS model

$$E_{\text{ue}_j,i}^{\text{comp}} = \gamma_{\text{ue}_j} (f_{\text{ue}_j,i})^2 \varphi_i^{\text{cycle}} \tag{16}$$

where γ_{ue_j} is the effective switching capacitance, $f_{\text{ue}_j,i}$ is the CPU frequency, and φ_i^{cycle} is the number of CPU cycles required by task t_i. If the task is offloaded to the UAV via SCU s_ℓ, the UE consumes transmission energy given by

$$E_{\text{ue}_j,s_\ell}^{\text{tx}}(t_i) = P_{\text{ue}_j}^{\text{tx}} \frac{\delta_i}{R_{\text{ue}_j,s_\ell}} \tag{17}$$

where $P_{\text{ue}_j}^{\text{tx}}$ is the transmit power, δ_i is the data size, and R_{ue_j,s_ℓ} is the achievable uplink rate.

2) UAV Energy Consumption: Each UAV uv_m consumes energy for executing offloaded tasks, communicating with SCUs and the BS, and for propulsion [34]. The total energy of UAV uv_m is as follows:

$$E_{\text{uv}_m}^{\text{total}} = E_{\text{uv}_m}^{\text{comp}} + E_{\text{uv}_m}^{\text{comm}} + E_{\text{uv}_m}^{\text{fly}}. \tag{18}$$

The computation energy for processing task t_i follows:

$$E_{\text{uv}_m,i}^{\text{comp}} = \gamma_{\text{uv}_m} (f_{\text{uv}_m,i})^2 \varphi_i^{\text{cycle}} \tag{19}$$

$$E_{\text{uv}_m}^{\text{comp}} = \sum_{t_i \in T_{\text{uv}_m}} E_{\text{uv}_m,i}^{\text{comp}} \tag{20}$$

while the communication energy for data exchange with SCU s_ℓ and BS is as follows:

$$E_{\text{uv}_m}^{\text{comm}} = P_{s_\ell}^{\text{tx}} \frac{\delta_{s_\ell,\text{uv}_m}}{R_{s_\ell,\text{uv}_m}} + P_{\text{uv}_m}^{\text{tx}} \frac{\delta_{\text{uv}_m,\text{BS}}}{R_{\text{uv}_m,\text{BS}}}. \tag{21}$$

The propulsion energy of rotary-wing UAV uv_m with flight speed v_{uv_m} and rotor-tip speed $v_{\text{uv}_m}^{\text{tip}}$ is given by

$$P_{\text{uv}_m}^{\text{fly}} = \eta_1 \left(1 + \frac{3v_{\text{uv}_m}^2}{(v_{\text{uv}_m}^{tip})^2}\right) + \eta_2 \left(\eta_3 + \frac{v_{\text{uv}_m}^4}{4} - \frac{v_{\text{uv}_m}^2}{2}\right) + \eta_4 v_{\text{uv}_m}^3$$

$$E_{\text{uv}_m}^{\text{fly}} = P_{\text{uv}_m}^{\text{fly}} \tau_{\text{uv}_m}^{\text{fly}}. \tag{22}$$

Here, $\tau_{\text{uv}_m}^{\text{fly}}$ is the duration of flight time. This unified energy model captures the contributions from all active network entities, forming the foundation for energy-aware offloading and trajectory optimization in the proposed UAV-assisted MEC framework.

F. Problem Formulation

The objective of the proposed UAV-assisted MEC framework is to minimize the maximum end-to-end latency among all tasks by jointly optimizing task offloading, CPU allocation, caching, and UAV scheduling variables, while satisfying resource, energy, and mobility constraints. The offloading relationship is represented by a binary variable $g_{\text{ue}_j,n}$, where $g_{\text{ue}_j,n} = 1$ if task t_i from UE ue_j is assigned to node $n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\}$, and 0 otherwise. The optimization problem is formulated as follows:

$$\underset{\substack{g_{\text{ue}_j,n}, h_{n,i}, \\ C_{n,P_i}^{\text{cache}}, q_{\text{uv}_m}}}{\text{minimize}} \ \max_{t_i \in T} \mathbb{L}(t_i) \tag{23a}$$

$$\text{subject to} \sum_{n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\}} g_{\text{ue}_j,n} = 1, \ \forall \text{ue}_j \in U, t_i \in T_{\text{ue}_j} \tag{23b}$$

$$\mathbb{L}(t_i) \leq D_i^{\max}, \ \forall t_i \in T \tag{23c}$$

$$\sum_{t_i \in T_n} h_{n,i} \leq 1, \ T_n = \{t_i : g_{\text{ue}_j,n} = 1\} \tag{23d}$$

$$g_{\text{ue}_j,n} \leq C_{n,P_i}^{\text{cache}}, \ \forall \text{ue}_j \in U, n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\} \tag{23e}$$

$$\sum_{t_i \in T_{\text{uv}_m}} g_{\text{ue}_j,\text{uv}_m} \leq M_{task,\text{uv}_m}^{\max}, \ \forall \text{uv}_m \in V \tag{23f}$$

$$E_{\text{ue}_j}^{\text{total}} \leq E_{\text{ue}_j}^{\max}, \ E_{\text{uv}_m}^{\text{total}} \leq E_{\text{uv}_m}^{\max}, \ \forall (\text{ue}_j, \text{uv}_m) \tag{23g}$$

$$q_{\text{uv}_m} \in [0, A] \times [0, A], \ \forall \text{uv}_m \in V \tag{23h}$$

$$\sum_{P_j \in P} C_{n,P_j}^{\text{cache}} l_j \leq C_n^{\max}, \quad \forall n \in \{\text{ue}_j, \text{uv}_m, \text{BS}\} \tag{23i}$$

$$d_{s_\ell,\text{uv}_m} \leq H \tan\theta \quad \text{if } g_{\text{ue}_j,\text{uv}_m} = 1. \tag{23j}$$

Here, $\mathbb{L}(t_i)$ denotes the end-to-end latency of task t_i, which includes the uplink transmission, computation, and result-delivery time components. Constraint (23b) enforces that each task is assigned to only one node, preventing redundant computation, while (23c) ensures that every task is completed within its deadline. Constraint (23d) limits the total CPU fraction allocated to all tasks on a node, maintaining feasible computation scheduling, and (23e) guarantees that tasks are executed only where the required programs are cached. The UAV task-handling capacity is bounded by (23f), which prevents overloading of airborne servers, whereas

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

(23g) ensures that both UEs and UAVs operate within their energy budgets, considering computation, communication, and propulsion costs. Constraint (23h) restricts UAV movement to the defined service area at a fixed altitude, maintaining reliable communication links, and (23i) ensures that caching decisions respect the finite memory capacities of UEs and UAVs. Constraint (23j) ensures that each offloading device remains within the effective communication range of its associated UAV.

IV. Theoretical Analysis

The formulated optimization problem (23) jointly minimizes the maximum end-to-end latency by optimizing three interdependent decision blocks: caching, offloading with CPU allocation, and UAV trajectory scheduling. The problem couples discrete caching/offloading decisions with continuous CPU-frequency and UAV-position variables, leading to a mixed-integer nonlinear programming (MINLP) formulation that is NP-hard. To achieve a tractable solution, an alternating block-coordinate optimization (BCO) approach is employed, where each block is optimized iteratively while the others are kept fixed. This decomposition ensures monotonic improvement in the objective function and convergence to a blockwise stationary point.

A. Problem Decomposition and Constraint Coupling

Let the global objective function be $\mathcal{F}(C^{\text{cache}}, G, H, Q) = \max_{t_i \in T} \mathbb{L}(t_i)$, representing the maximum end-to-end latency across all tasks, subject to constraints (23b)–(23j). The decision variables are partitioned into three independent but coupled subproblems, as follows.

1) *Caching Block (C^{cache})*: It updates UE/UAV local repositories, constrained by limited storage capacity (23i).
2) *Offloading and CPU Block (G, H)*: It assigns tasks and allocates CPU shares under the feasibility conditions (23b)–(23g).
3) *UAV Trajectory Block (Q)*: It updates UAV positions within their operational domain, respecting mobility and coverage limits (23h)–(23j).

Each subproblem is optimized sequentially, ensuring that the objective function does not increase at each update.

B. BCO and Convergence

Let $(C_r^{\text{cache}}, G_r, H_r, Q_r)$ denote the variables at iteration r. The update sequence proceeds as follows.

1) *Caching Update*: For fixed (G_r, H_r, Q_r)

$$\mathcal{F}(C_{r+1}^{\text{cache}}, G_r, H_r, Q_r) \le \mathcal{F}(C_r^{\text{cache}}, G_r, H_r, Q_r). \tag{24}$$

2) *Offloading and CPU Update*: For fixed $(C_{r+1}^{\text{cache}}, Q_r)$

$$\mathcal{F}(C_{r+1}^{\text{cache}}, G^{r+1}, H^{r+1}, Q_r) \le \mathcal{F}(C_{r+1}^{\text{cache}}, G_r, H_r, Q_r). \tag{25}$$

3) *Trajectory Update*: For fixed $(C_{r+1}^{\text{cache}}, G^{r+1}, H^{r+1})$

$$\mathcal{F}(C_{r+1}^{\text{cache}}, G^{r+1}, H^{r+1}, Q^{r+1}) \le \mathcal{F}(C_{r+1}^{\text{cache}}, G^{r+1}, H^{r+1}, Q_r). \tag{26}$$

By transitivity, after each full iteration, we have the following:

$$\mathcal{F}_{r+1} \le \mathcal{F}_r, \qquad \forall r \tag{27}$$

where $\mathcal{F}_r := \mathcal{F}(C_r^{\text{cache}}, G_r, H_r, Q_r)$. Since $\mathcal{F}$ is lower bounded (latency is always nonnegative) and the feasible set is compact due to the system constraints, the sequence $\{\mathcal{F}_r\}$ is monotonically nonincreasing and convergent. The algorithm terminates when $|\mathcal{F}_{r+1} - \mathcal{F}_r| \le \varepsilon$ for a small threshold $\varepsilon \ge 0$. This guarantees algorithmic stability and practical implementability. Such convergence is standard for block-coordinate descent methods in mixed-integer nonconvex optimization [12].

C. Caching Optimization Subproblem

For fixed (G, H, Q), the caching subproblem determines which programs should be stored locally on UEs and UAVs to maximize cache-hit probability and reduce backhaul dependence. Each program P_j is assigned a composite priority score that jointly accounts for two factors: 1) its frequency of necessity F_{P_j}, in the current context (how often the program is requested by active tasks); and 2) its program weight W_{P_j}, which quantifies the program's effectiveness based on the number of tasks it has successfully completed in previous iterations. The score computation model is formulated as follows:

$$F_{P_j} = \begin{cases} 0.6 \times \dfrac{N_{P_j,T}}{|P_{\text{needed}}|}, & \text{if} |P_{\text{needed}}| > 0 \\ 0, & \text{otherwise} \end{cases} \tag{28}$$

$$W_{\text{score},P_j} = \begin{cases} 0.4 \times \dfrac{W_{P_j}}{N_{\text{tasks}}}, & \text{if } N_{\text{tasks}} > 0 \\ 0, & \text{otherwise.} \end{cases} \tag{29}$$

Here, $N_{P_j,T}$ denotes the number of currently active tasks requiring program P_j, and $P_{\text{needed}} = \{P_j \in P \mid \exists t_i \in T \text{ that requires } P_j\}$ is the set of all requested programs in the current scheduling window. The term W_{P_j} represents the accumulated weight of program P_j, reflecting its prior utility based on the number of tasks it has served successfully, while N_{tasks} is the total number of ongoing tasks. The overall caching priority of each program is expressed as $S_{P_j} = F_{P_j} + W_{\text{score},P_j}$, and the highest scoring programs are cached until the storage constraint in (14) is satisfied.

Finally, the updated cache state for node n is defined as

$$C_n^{\text{cache}} = C_{\text{top}}^{\text{cache}} \cup C_{\text{add}}^{\text{cache}} \tag{30}$$

where $C_{\text{top}}^{\text{cache}}$ includes the highest ranked programs according to S_{P_j}, and $C_{\text{add}}^{\text{cache}}$ includes additional randomly selected programs if residual storage space remains. This adaptive caching strategy enhances cache utilization, increases hit probability, and reduces the offloading and backhaul delay in dynamic MEC environments.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

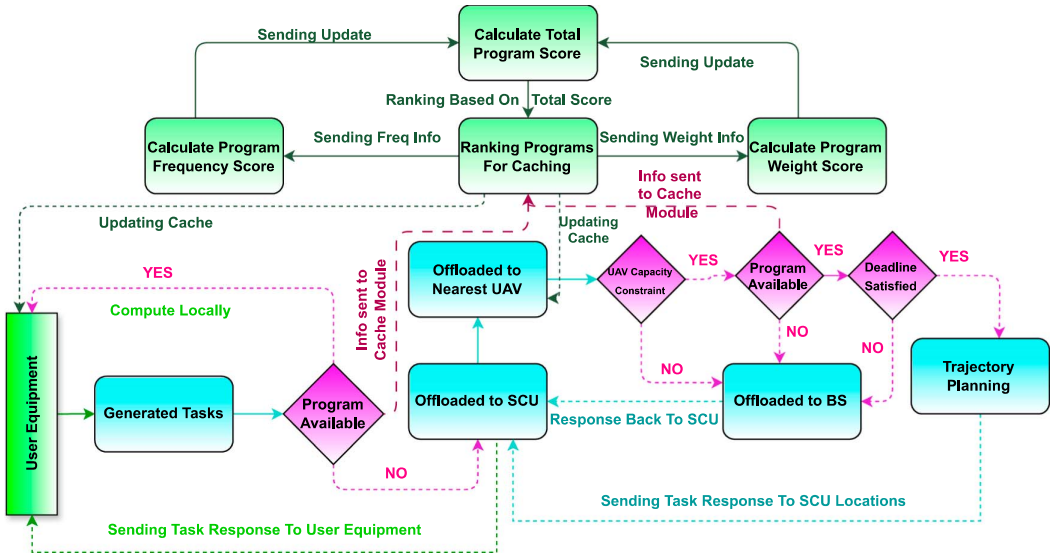


Fig. 2. Workflow of the proposed joint caching, offloading, and trajectory optimization framework.

D. Offloading and Resource Allocation Subproblem

Given the caching state (C^{cache}) and UAV positions (Q), the offloading and resource-allocation block determines the optimal execution node for each task and assigns suitable CPU resources to minimize the end-to-end latency. Each UE-generated task $t_i = (\delta_i, P_i, \varphi_i^{\text{cycle}}, D_i^{\max})$ can be processed either locally, at a UAV, or at the BS, depending on cache availability, energy constraints, and computational capacity. The objective is to minimize the total completion delay $\mathbb{L}(t_i)$ under the feasibility constraints (23b)–(23j). For each feasible node n, both latency and energy consumption are evaluated, and the node with minimum $\mathbb{L}(t_i)$ is selected as the optimal execution point n^*. Once n^* is determined, the binary indicator g_{ue_j,n^*} is updated, and CPU resources are allocated according to the node's available capacity. The CPU-share allocation satisfies (23d), with optimal proportion

$$h^{\star}_{n,i} = \frac{\sqrt{\varphi_i^{\text{cycle}}}}{\sum_{k\in T_n}\sqrt{\varphi_k^{\text{cycle}}}} \tag{31}$$

$$f^{\star}_{n,i} = h^{\star}_{n,i} F_n^{\max} \tag{32}$$

ensuring that computational capacity $F_n^{\max}$ is distributed among tasks in proportion to their computational workloads φ_i^{cycle}.

E. UAV Trajectory Optimization Subproblem

Given (C^{cache}, G, H), the goal of this subproblem is to determine an efficient visiting order for each UAV uv_m such that all associated SCUs are served while minimizing flight distance and satisfying the UAV's propulsion-energy constraint. Instead of solving a continuous nonconvex optimization over UAV coordinates, a low-complexity nearest neighbor heuristic is adopted. Each UAV incrementally constructs its flight path by visiting the closest unserved SCU, thereby reducing total travel distance and delivery delay.

Formally, the trajectory design can be expressed as a discrete path-planning problem

$$\min_{Q_{\text{uv}_m}} \sum_{s_\ell \in Q_{\text{uv}_m}} \|q_{\text{uv}_m} - q_{s_\ell}\|_2 \tag{33}$$

$$\text{subject to } E^{\text{fly}}_{\text{uv}_m} \le E^{\max}_{\text{uv}_m} \tag{34}$$

where Q_{uv_m} is the ordered set of SCUs visited by UAV uv_m, $\|q_{\text{uv}_m} - q_{s_\ell}\|_2$ denotes the Euclidean distance between UAV and SCU, and $E^{\text{fly}}_{\text{uv}_m}$ is the propulsion energy consumed along the trajectory.

V. Proposed Methodology

This section presents the proposed solution framework for solving the joint optimization problem (23), which aims to minimize the end-to-end latency in the UAV-assisted MEC system. Due to the mixed-integer nonlinear nature of (23), directly obtaining an optimal solution is computationally intractable. To overcome this, we decompose the problem into three interdependent submodules: 1) caching decision; 2) offloading and resource allocation; and 3) UAV trajectory and task delivery planning. Each submodule is iteratively optimized until convergence. The overall workflow of the proposed framework is illustrated in Fig. 2.

A. Caching Decision Module

The caching decision module dynamically determines which programs should be stored at UEs and UAVs based on task frequency and program utilization. Since the caching decisions directly influence the feasibility of task execution and offloading, this module serves as the foundation for subsequent optimization steps. Each program $p_r \in P$ is assigned two metrics: a frequency score F_{p_r} representing the number of times it is requested within the current time window and a weight score W_{score,p_r} denoting its historical importance (e.g., based on task

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Dynamic Program Caching Algorithm.

Input: Program set P, request frequency F_{p_r}, weight score W_{p_r}, cache capacities $C_{UE}^{cache}, C_{UAV}^{cache}$
Output: Updated cache matrix C_{n,p_r}^{cache} for all $n \in \{ue_j, uv_m\}$

foreach $p_r \in P$ **do**
  Compute $S_{p_r} = F_{p_r} + W_{score,p_r}$;
Rank all $p_r \in P$ in descending order of S_{p_r};
Select top-k programs under each node's capacity constraint;
Update C_{n,p_r}^{cache} accordingly;

completion statistics). The overall caching priority score S_{p_r} is computed as a weighted sum

$$S_{p_r} = F_{p_r} + W_{\text{score},p_r}. \tag{35}$$

Programs are ranked in descending order of S_{p_r}, and the top programs are selected for caching at each node, subject to its storage constraint given in (14). This dynamic cache update ensures that frequently requested programs remain locally accessible, reducing backhaul dependency and overall communication latency.

B. Offloading and Resource Allocation Module

The second submodule determines the optimal offloading decisions and computational resource allocations, given the current caching and network states. Each UE decides whether to execute a task locally, offload it to a nearby UAV, or forward it to the BS based on minimizing latency $\mathbb{L}(t_i)$ under the deadline and energy constraints defined in (23). The offloading decision variable $g_{\text{ue}_j,n}$ and CPU allocation fraction h_{n,t_i} are jointly optimized according to the following objective for each task:

$$\min_{g_{\text{ue}_j,n}, h_{\text{ue}_j,i}} \quad \mathbb{L}(t_i) \tag{36}$$

subject to the constraints on offloading uniqueness, cache availability, and computational capacity. The procedure is described in Algorithm 2.

This algorithm adaptively balances task load across local, aerial, and cloud computing layers, ensuring latency-energy tradeoff compliance and respecting UAV task capacity constraints.

Algorithm 2: Offloading and Resource Allocation.

Input: Task set T, caching matrix C^{cache}, CPU limits $F_n^{\max}$, energy budgets $E_{ue_j}^{\max}, E_{uv_m}^{\max}$
Output: Offloading matrix $g_{ue_j,n}$, CPU allocations h_{n,t_i}

foreach *task* $t_i = (\delta_i, P_i, \varphi_i^{cycle}, D_i^{\max}) \in T$ **do**
  Identify candidate nodes $n \in \{ue_j, uv_m, BS\}$ satisfying $C_{n,P_i}^{cache} = 1$;
  Compute total latency $\mathbb{L}(t_i)$ and energy $E_n^{total}(t_i)$ for each feasible n;
  Select node $n^* = \arg\min_n \mathbb{L}(t_i)$;
  Set $g_{ue_j,n^*} = 1$ and allocate CPU share h_{n^*,t_i} using available $F_{n^*}^{\max}$;

Algorithm 3: UAV Trajectory and Delivery Optimization.

Input: UAV set V, SCU set S, UAV positions q_{uv_m}, SCU positions q_{s_ℓ}, flight-energy limits $E_{uv_m}^{\max}$
Output: Optimized UAV trajectories Q_{uv_m}

foreach *UAV* $uv_m \in V$ **do**
  Initialize $Q_{uv_m} = \emptyset$, $E_{uv_m}^{fly} = 0$;
  while $E_{uv_m}^{fly} < E_{uv_m}^{\max}$ ***and*** *SCUs remain unserved* **do**
    Select nearest unvisited SCU: $s_\ell^* = \arg\min_{s_\ell \in S_{unserved}} \|q_{uv_m} - q_{s_\ell}\|_2$;
    Append $q_{s_\ell^*}$ to Q_{uv_m};
    Update position $q_{uv_m} \leftarrow q_{s_\ell^*}$;
    Update flight energy $E_{uv_m}^{fly} \leftarrow E_{uv_m}^{fly} + \eta_d \|q_{uv_m} - q_{s_\ell^*}\|_2$;
    Mark s_ℓ^* as served;

C. UAV Trajectory and Task Delivery Planning

After task allocation, each UAV $\text{uv}_m \in V$ plans its flight trajectory to sequentially visit the associated SCUs and deliver processed results to the corresponding UEs. The UAV trajectory optimization problem aims to minimize the total service and delivery time while satisfying propulsion energy and coverage constraints. The trajectory of UAV uv_m is represented as a sequence of SCU locations

$$Q_{\text{uv}_m} = \{q_{s_1}, q_{s_2}, \ldots, q_{s_L}\} \tag{37}$$

where $q_{s_\ell} = (x_{s_\ell}, y_{s_\ell})$ denotes the position of SCU s_ℓ visited by UAV uv_m. At each iteration, UAV uv_m selects its next destination SCU according to the nearest neighbor criterion

$$s_\ell^* = \arg \min_{s_\ell \in S_{unserved}} \|q_{\text{uv}_m} - q_{s_\ell}\|_2 \tag{38}$$

subject to the residual propulsion-energy constraint $E_{\text{uv}_m}^{\text{fly}} \leq E_{\text{uv}_m}^{\max}$. After visiting the selected SCU, the UAV updates its trajectory and cumulative energy consumption as follows:

$$Q_{\text{uv}_m} \leftarrow Q_{\text{uv}_m} \cup \{q_{s_\ell^*}\} \tag{39}$$

$$E_{\text{uv}_m}^{\text{fly}} \leftarrow E_{\text{uv}_m}^{\text{fly}} + \eta_d \|q_{\text{uv}_m} - q_{s_\ell^*}\|_2 \tag{40}$$

where η_d denotes the energy cost per unit distance traveled. The UAV repeats this process until all assigned SCUs are served or the maximum flight-energy budget $E_{\text{uv}_m}^{\max}$ is reached. The total task-delivery time includes both the UAV travel delay and the downlink communication delay from UAV to SCU to UE. The detailed steps are summarized in Algorithm 3.

By iteratively optimizing caching, offloading, and trajectory planning modules, the proposed hybrid framework adaptively adjusts to varying network states and workload distributions as

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

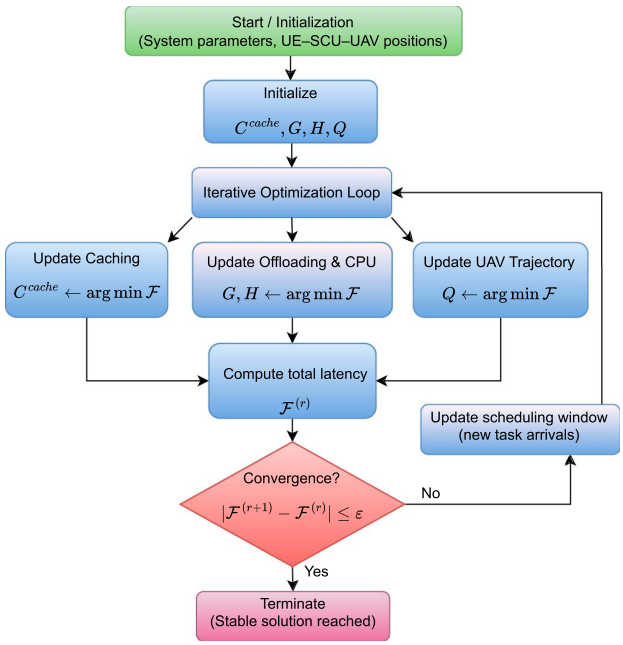


Fig. 3. Algorithmic pipeline of the proposed UAV-assisted MEC framework.

shown in Fig. 3. The resulting joint optimization ensures low latency, efficient energy utilization, and improved scalability in dynamic UAV-assisted MEC environments.

D. Complexity Analysis

We provide a brief complexity analysis for the major algorithms in the proposed framework. All calculations follow from the previously stated system model and assumptions.

1) Caching Optimization: The caching algorithm has space complexity $O(N_{\text{progs}} + N_{\text{tasks}})$ and updates program frequencies and scores in $O(N_{\text{progs}})$. Sorting all programs incurs $O(N_{\text{progs}} \log N_{\text{progs}})$, and selecting top programs for each entity (UE or UAV) requires $O(N_{\text{progs}} \times N_{\text{entity}})$. Updating cached programs incurs $O(M \times N_{\text{entity}})$, where M is the number of selected top programs. Thus, the total time complexity is, $O\left(N_{\text{progs}} \log N_{\text{progs}} + N_{\text{progs}} N_{\text{entity}}\right)$.

2) Offloading Decision: The offloading algorithm iterates over all UEs and, for each, searches through all SCUs and UAVs to assign tasks, resulting in, $O\left(N_{\text{UE}}(N_{\text{SCU}} + N_{\text{UAV}})\right)$.

3) Task Delivery and Trajectory Planning: Task delivery iterates over all tasks and associates each with an SCU, yielding $O(N_{\text{tasks}} + N_{\text{SCU}} N_{\text{tasks}})$. UAV trajectory planning requires a linear search over all SCUs for each UAV, i.e., $O(N_{\text{SCU}} N_{\text{UAV}})$. The combined complexity is, therefore, $O\left(N_{\text{SCU}}(N_{\text{UAV}} + N_{\text{tasks}}) + N_{\text{tasks}}\right)$.

4) Overall System Complexity: The overall time complexity for the joint framework, accounting for all major components is, $O(N_{\text{progs}} \log N_{\text{progs}} + N_{\text{progs}} N_{\text{entity}} + N_{\text{UE}}(N_{\text{SCU}} + N_{\text{UAV}}) + N_{\text{SCU}}(N_{\text{UAV}} + N_{\text{tasks}}) + N_{\text{tasks}})$. This polynomial complexity confirms the tractability and scalability of the proposed approach for practical MEC network sizes.

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Number of UEs (N_{UE})	50–95
Number of SCUs (N_{SCU})	10–20
Number of UAVs (N_{UAV})	5–20
Simulation Area Size	$200 \times 200\,\text{m}^2$
Program Capacity of UE ($C_{\text{ue}_j}^{\max}$)	4
Program Capacity of UAV ($C_{\text{uv}_m}^{\max}$)	10
Task Capacity of UAV ($M_{\text{Task,UAV}}$)	15
Number of Tasks (N_{tasks})	30
Number of Programs (N_{progs})	20
Bandwidth per UAV	60 MHz
Movement per UE per iteration	5 m
UAV Velocity (v_{uv_m})	10 m/s
Height of UAV (H)	60 m
Time Unit (per SCU travel in minutes)	1

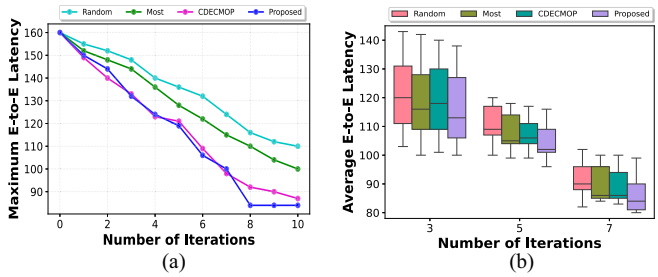


Fig. 4. Convergence analysis with varying number of iterations: (a) maximum E-to-E latency; and (b) average E-to-E latency.

VI. NUMERICAL ANALYSIS

This section evaluates the performance of the proposed UAV-assisted MEC framework and compares it with three benchmark schemes: random caching, most caching [35], and the EMO-TDL-SA framework [36]. The EMO-TDL-SA model integrates simulated annealing (SA) and target distribution learning (TDL) into an evolutionary multiobjective DRL (EMODRL) framework for UAV-assisted MEC optimization. All simulations were conducted on a workstation equipped with an Intel Core i9 (12th generation) processor, 16 GB DDR5 RAM, and an NVIDIA GeForce RTX GPU to ensure accurate computation and reproducibility. The system parameters used in this study are summarized in Table II. We are providing brief information about the reference caching algorithms that we have mentioned above.

The comparative algorithms are summarized as follows.

1) *Random Caching (Random):* The random caching algorithm is the method of assigning programs to each of the UE and UAV in the network in a completely random fashion, so that a wide variety of programs are present at different locations, keeping storage capacity as a constraint.
2) *Most Caching (Most):* The most caching algorithm is a selective caching strategy that selects the most frequent programs for the cache. It updates the cache so that, based on present demand, the most demanded programs are cached.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

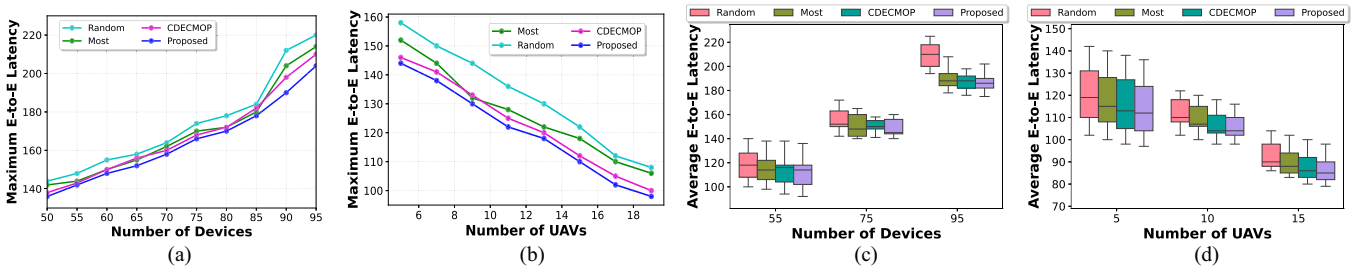


Fig. 5. Performance comparison of: (a) maximum E-to-E latency versus number of devices; (b) Maximum E-to-E latency versus number of UAVs; (c) average E-to-E latency versus number of devices; and (d) average E-to-E latency versus number of UAVs.

3) *EMO-TDL-SA (CDECMOP):* An EMODRL framework that jointly minimizes task delay and UAV energy consumption using SA-based exploration and TDL-guided policy updates to achieve Pareto-optimal performance. For a fair comparison, we assume calculation delay and energyconsumption multiobjective optimization problem (CDECMOP) also supports UE caching and local processing; however, it lacks adaptive cache coordination and iterative task-trajectory refinement, which limits its responsiveness under dynamic MEC conditions.

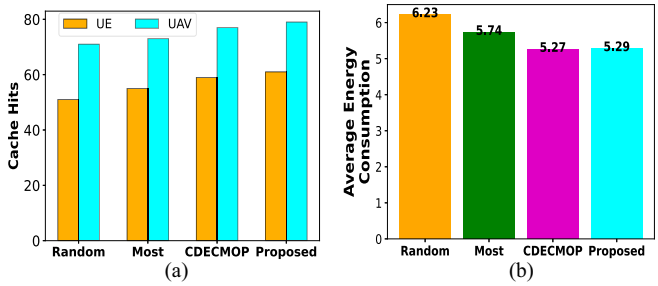


Fig. 6. Performance comparison across algorithms: (a) cache hit ratio at UEs and UAVs; and (b) average energy consumption by UAVs.

A. Simulation Results

Below, we outline the parameters used to evaluate the performance of our method. The reference parameters are consistent with [12]. The primary performance metric is the maximum task latency among all devices, measured over multiple consecutive iterations. This metric is assessed for various network scenarios by varying the number of UEs (devices) and UAVs. A brief summary is provided below.

1) Optimization Results: Fig. 4 illustrates the convergence behavior of the proposed scheme compared to random, most, and CDECMOP algorithms in terms of maximum and average end-to-end (E-to-E) latency. As observed in Fig. 4(a), the proposed method achieves rapid convergence, reducing the maximum E-to-E latency to approximately 84 s by iteration 7. In contrast, CDECMOP converges around 88 s, while most and random caching schemes stabilize near 100 and 110 s, respectively. This represents an improvement of nearly 16% over most caching and 23% over random caching, demonstrating faster optimization and better delay control. Similarly, Fig. 4(b) shows the average E-to-E latency trend across iterations. The proposed approach consistently achieves lower average latency values compared to other algorithms, reflecting its efficient offloading, caching, and scheduling synergy. At iteration 7, the proposed scheme records an average latency between 80 and 95 s, outperforming CDECMOP (85–98 s), most caching (90–105 s), and random caching (95–110 s). These results confirm that the proposed method ensures faster convergence and superior overall performance through coordinated caching and trajectory-aware offloading optimization.

2) Algorithm Performance Comparison: Fig. 5 presents the comparative performance analysis of the proposed framework against CDECMOP, most caching, and random caching algorithms under different network configurations. As shown in Fig. 5(a), when the number of devices increases from 50 to 95, the maximum E-to-E latency rises across all methods due to increased task load and limited computational resources. However, the proposed method consistently achieves lower latency, reaching approximately 202 s at 95 devices, compared to 208 s for CDECMOP, 214 s for most caching, and 220 s for random caching. This improvement of 2.9% over CDECMOP and up to 8.2% over random caching highlights the superior load adaptability of the proposed strategy. In Fig. 5(b), as the number of UAVs increases from 5 to 19, the system gains higher task parallelism, reducing end-to-end latency. The proposed approach achieves the lowest maximum latency of approximately 98 s at 19 UAVs, outperforming CDECMOP (100 s), most caching (106 s), and random caching (108 s). This indicates that the proposed model leverages UAV coordination and caching efficiency more effectively than its competitors. Fig. 5(c) shows that the average E-to-E latency increases with the number of devices due to accumulated task requests. Even under heavier loads, the proposed framework maintains better latency stability, achieving an average of 182 s at 95 devices, 7.5% lower than CDECMOP and up to 18.6% lower than random caching. The integration of caching and computation scheduling enables the system to sustain efficient resource utilization. Finally, Fig. 5(d) illustrates that with an increasing number of UAVs, all algorithms experience latency reduction due to improved task distribution. The proposed framework achieves the lowest latency range of 80–95 s, followed by CDECMOP (85–100 s), most caching (90–105 s), and random caching (95–110 s). Overall, these results confirm that while CDECMOP demonstrates competitive performance, the proposed framework outperforms it across all scenarios by offering faster convergence, better scalability, and more stable latency control

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

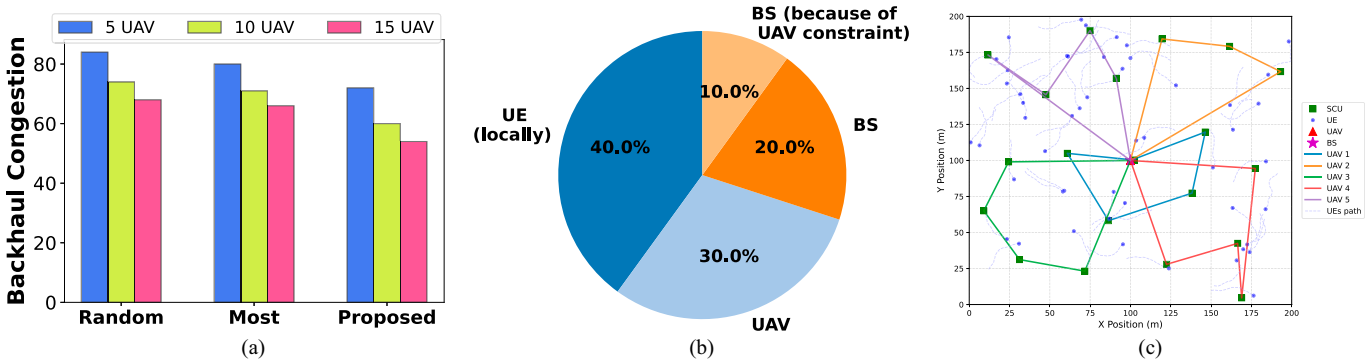


Fig. 7. Comparative analysis under varying conditions: (a) backhaul congestion vs UAV count; (b) task completion probability by location; and (c) adaptive UAV trajectories for task delivery.

TABLE III
ANOVA TEST FOR PERFORMANCE METRICS.

Method	Source of Variation	SS	*df*	MS	F static	P value	F critical
Caching Performance	Between Groups	1.761905	2	0.880952	0.407282	0.668254	3.238096
	Within Groups	84.35714	39	2.163004	-	-	-
	Total	86.11905	41	-	-	-	-
Max Task Latency	Between Groups	8.047619	2	4.023810	1.249716	0.297809	3.238096
	Within Groups	125.5714	39	3.219780	-	-	-
	Total	133.6190	41	-	-	-	-

through joint caching, offloading, and trajectory-aware resource coordination.

Fig. 6 represents a comparison of the proposed UAV-assisted MEC framework in terms of cache efficiency and energy utilization against baseline algorithms. As shown in Fig. 6(a), the proposed caching mechanism achieves the highest cache hit rates, 61 at UEs and 79 at UAVs, representing improvements of approximately 20% and 11% over random caching and slightly surpassing CDECMOP. This improvement arises from the adaptive cache update mechanism that jointly accounts for program popularity and task relevance, minimizing redundant retrievals and reducing backhaul dependence. In Fig. 6(b), CDECMOP records a slightly lower average UAV energy consumption of $5.27\,\text{kJ}$ compared to $5.29\,\text{kJ}$ for the proposed framework. This comparison assumes that CDECMOP also incorporates UE caching and local task processing for a fair evaluation. The slightly lower energy usage of CDECMOP results from its evolutionary multiobjective optimization design, which explicitly balances energy and delay objectives, leading to more energy-conservative UAV scheduling. In contrast, the proposed framework emphasizes latency reduction through adaptive cache coordination and iterative trajectory refinement, causing marginally higher UAV energy expenditure while achieving superior cache efficiency, faster response, and overall system robustness under dynamic network conditions.

3) Robustness Testing: Fig. 7 presents a robustness evaluation of the proposed UAV-assisted MEC framework under different operational scenarios. In Fig. 7(a), the proposed scheme consistently exhibits reduced backhaul congestion as the number of UAVs increases from 5 to 15. The improvement is attributed to distributed caching and localized task execution, leading to a reduction of nearly 25% in backhaul load compared to random caching and 15% compared to most caching. Fig. 7(b) shows the task completion probability across different locations, where 40% of tasks are executed locally at UEs, 30% at UAVs, and 20% at the BS, while only 10% are processed at the BS due to UAV capacity constraints. This distribution demonstrates the system's ability to dynamically balance computational loads and maximize edge resource utilization. Fig. 7(c) illustrates the adaptive trajectories of UAVs and UEs during the optimization process. Each UAV initially departs from the BS and progressively moves toward the nearest SCU, guided by task distribution, cache availability, and residual capacity. As iterations progress, UAVs dynamically traverse between SCUs to collect and deliver computation tasks and subsequently return to the BS upon completing their assigned missions. This coordinated movement effectively minimizes communication distance, task latency, and overall energy consumption while ensuring efficient service delivery across the network. Overall, these results confirm that the proposed framework enhances network stability, reduces backhaul dependency, and maintains energy-efficient UAV operations under dynamic MEC environments.

B. Statistical Analysis

To analyze the performance of our *JOINT* framework, we have made a hypothesis test based on the numerical results obtained through analysis of variance (ANOVA), and it is shown in Table III. ANOVA test is used to estimate discrepancies among groups. We conducted a one-way ANOVA test among different groups to determine whether the null hypothesis should be rejected. The essential parameters for ANOVA test are, namely, *alternative hypothesis*, $\mathbb{H}_a$, and *null hypothesis*, $\mathbb{H}_o$. Null hypothesis helps in establishing equivalency among the groups like, $\mathbb{H}_o$: $\left\{\mathbb{G}_1^\delta = \mathbb{G}_2^\delta = \ldots = \mathbb{G}_3^\delta\right\}$. Similarly, the alternative hypothesis considers discrepancy among individual grouped pairs, like $\mathbb{H}_a$: $\left\{\exists(\mathbb{G}_i^\delta, \mathbb{G}_j^\delta), \mathbb{G}_i^\delta \neq \mathbb{G}_j^\delta\right\}$. For analyzing performance, we have considered α level as 0.05, where

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

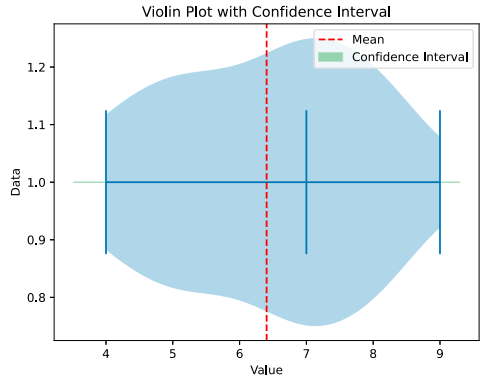


Fig. 8. CI for caching performance.

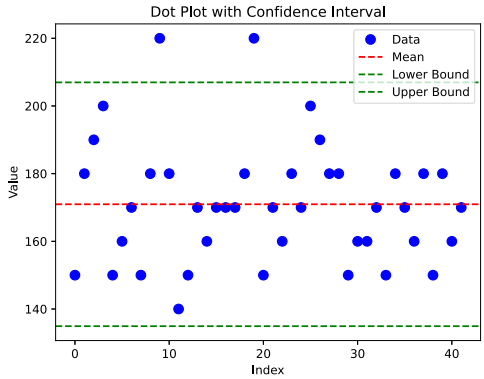


Fig. 9. CI for maximum end-to-end latency.

$(1-\alpha)$ determines the probability of accepting $\mathbb{H}_o$ hypothesis known as confidence interval (CI). Our existing algorithms are analyzed based on caching performance and task latency.

For the caching performance, it is visible that the F static value being 0.407282 is much less than the $F-$critical value 3.238096. The $P-value$ 0.668254 being greater than 0.05 indicates no significant difference between the groups at 5% significance level. In Fig. 8, the CI plot for caching performance is shown. From observing the plot, it is evident that it follows a normal distribution with a central tendency around the mean. The plot indicates a balanced distribution of values in which the majority of data points are concentrated around the mean.

For the maximum task latency, we see that the static value of F is 1.249716, which is also less than the critical value of F, and the $P-$value is 0.297809, which is again greater than 0.05, indicating no significant difference between the groups at the significance level 5%. In Fig. 9, the CI plot for maximum end-to-end latency is shown. From the plot, it is evident that the majority of data points fall within the CI, which indicates that the sample mean is a good estimate of the population mean. The plot shows a detailed visualization of data variability and central tendency. In both cases, the high $P-$values and lower F static (than critical ones) support the null hypothesis that the group means are almost similar for both metrics.

VII. Conclusion

A unified framework for joint service caching, task offloading, and service delivery in UAV-assisted MEC networks was established. Leveraging adaptive frequency and weight-based caching, resource-aware offloading, and UAV trajectory optimization, the proposed approach achieves substantial reductions in maximum task latency and backhaul congestion, consistently outperforming baseline strategies across diverse network scenarios. These results confirm the practical effectiveness of UAVs as agile edge computing nodes for scalable IoT applications. Building on this foundation, future work will extend the framework to dynamic wireless environments with unpredictable task arrivals and changing network conditions. Integrating federated and RL will enable collaborative, adaptive caching and offloading in partially observable settings. Additional enhancements will focus on supporting heterogeneous service types, detailed energy modeling, and robust security-aware scheduling. By combining adaptive optimization with collaborative intelligence, this research paves the way for the next generation of scalable, reliable, and autonomous UAV-assisted MEC solutions.

References

[1] R. Singh, R. Sukapuram, and S. Chakraborty, "A survey of mobility-aware multi-access edge computing: Challenges, use cases and future directions," *Ad Hoc Netw.*, vol. 140, 2023, Art. no. 103044.

[2] J. Wang et al., "Optimizing 6G integrated sensing and communications via expert networks," *IEEE Wireless Commun.*, vol. 32, no. 5, pp. 142–150, Oct. 2025.

[3] W. Chu, X. Jia, Z. Yu, J. C. Lui, and Y. Lin, "Joint service caching, resource allocation and task offloading for MEC-based networks: A multi-layer optimization approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2958–2975, Apr. 2023.

[4] C. Fang et al., "Joint task offloading and content caching for NOMA-aided cloud-edge-terminal cooperation networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 15586–15600, Oct. 2024.

[5] X. Dai, Z. Xiao, H. Jiang, and J. C. Lui, "UAV-assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2024.

[6] W. Zhu, X. Deng, J. Gui, H. Zhang, and G. Min, "Cost-effective task offloading and resource scheduling for mobile edge computing in 6G space-air-ground integrated network," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 19428–19442, Jun. 2025.

[7] A. Al-Hilo, M. Samir, C. Assi, S. Sharafeddine, and D. Ebrahimi, "UAV-assisted content delivery in intelligent transportation systems-joint trajectory planning and cache management," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5155–5167, Aug. 2021.

[8] L. He, et al., "Space-air-ground integrated MEC-assisted industrial cyber-physical systems: An online decentralized optimization approach," 2024, *arXiv:2411.09712v1*.

[9] G. Sun, J. Wu, and Z. Sun, "Collaborative computation offloading, caching, communication, computation, and trajectory control in UAV-assisted MEC system," in *Proc. ACM Turing Award Celebration Conf.-China*, 2024, pp. 203–204.

[10] G. Sun et al., "JC^{5}A: Service delay minimization for aerial MEC-assisted industrial cyber-physical systems," *IEEE Trans. Services Comput.*, vol. 18, no. 5, pp. 2976–2993, Sep./Oct. 2025.

[11] W. Fan, F. Xiao, Y. Pan, X. Chen, L. Han, and S. Yu, "Latency-aware joint task offloading and energy control for cooperative mobile edge computing," *IEEE Trans. Services Comput.*, vol. 18, no. 3, pp. 1515–1528, May/Jun. 2025.

[12] G. Zheng, C. Xu, M. Wen, and X. Zhao, "Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10934–10947, Oct. 2022.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.

[13] J. Poorvi, A. Kalita, and M. Gurusamy, "Reliable and efficient data collection in UAV based IoT networks," *IEEE Commun. Surveys Tuts.*, early access, Mar. 2025, doi: 10.1109/COMST.2025.3550274.
[14] G. Colajanni, P. Daniele, A. Nagurney, L. S. Nagurney, and D. Sciacca, "A three-stage stochastic optimization model integrating 5G technology and UAVs for disaster management," *J. Global Optim.*, vol. 86, no. 3, pp. 741–780, 2023.
[15] A. F. Moiz, H. M. Khawaja, Z. H. Khan, and R. Sohail, "QuadSWARM: A real-time autonomous surveillance system using multi-quadcopter UAVs," in *Proc. 1st Int. Conf. Adv. Innovations Smart Cities (ICAISC)*, IEEE, 2023, pp. 1–6.
[16] Y. Liu, C. Yang, X. Chen, and F. Wu, "Joint hybrid caching and replacement scheme for UAV-assisted vehicular edge computing networks," *IEEE Trans. Intell. Veh.*, vol. 9, no. 1, pp. 866–878, Jan. 2024.
[17] S. Akter, D. Van Anh Duong, and S. Yoon, "Joint optimization of AAV trajectory, task offloading, and resource allocation in AAV-aided emergency response operations," *IEEE Internet Things J.*, vol. 12, no. 12, pp. 21944–21959, Jun. 2025.
[18] N. Yang, S. Chen, H. Zhang, and R. Berry, "Beyond the edge: An advanced exploration of reinforcement learning for mobile edge computing, its applications, and future research trajectories," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 1, pp. 546–594, Feb. 2025.
[19] M. Sheng, Y. Dai, J. Liu, N. Cheng, X. Shen, and Q. Yang, "Delay-aware computation offloading in NOMA MEC under differentiated uploading delay," *IEEE Trans. Wireless Commun.*, vol. 19, no. 4, pp. 2813–2826, Apr. 2020.
[20] C. Xu, G. Zheng, and X. Zhao, "Energy-minimization task offloading and resource allocation for mobile edge computing in NOMA heterogeneous networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 16001–16016, Dec. 2020.
[21] T. Zhang, Y. Xu, J. Loo, D. Yang, and L. Xiao, "Joint computation and communication design for UAV-assisted mobile edge computing in IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 8, pp. 5505–5516, Aug. 2019.
[22] R. Zhou, X. Wu, H. Tan, and R. Zhang, "Two time-scale joint service caching and task offloading for UAV-assisted mobile edge computing," in *Proc. IEEE INFOCOM or IEEE Conf. Comput. Commun.*, Piscataway, NJ, USA: IEEE, 2022, pp. 1189–1198,
[23] Q. Wei, R. Li, W. Bai, and Z. Han, "Multi-UAV-enabled energy-efficient data delivery for low-altitude economy: Joint coded caching, user grouping, and UAV deployment," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 27519–27532, Jul. 2025.
[24] B. Jiang, J. Yang, H. Xu, H. Song, and G. Zheng, "Multimedia data throughput maximization in Internet-of-Things system based on optimization of cache-enabled UAV," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3525–3532, Apr. 2019.
[25] M. Yasir, S. K. Uz Zaman, T. Maqsood, F. Rehman, and S. Mustafa, "CoPUP: Content popularity and user preferences aware content caching framework in mobile edge computing," *Cluster Comput.*, vol. 26, no. 1, pp. 267–281, 2023.
[26] X. Gao, Y. Sun, H. Chen, X. Xu, and S. Cui, "Joint computing, pushing, and caching optimization for mobile edge computing networks via soft actor-critic learning," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9269–9281, Mar. 2024.
[27] Z. Li et al., "Delay efficient caching enabled hierarchical mobile edge computing networks," *IEEE Trans. Commun.*, vol. 73, no. 10, pp. 9087–9101, Oct. 2025.
[28] S. Yoo, S. Jeong, J. Kim, and J. Kang, "Cache-assisted mobile-edge computing over space–air–ground integrated networks for extended reality applications," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18306–18319, May 2024.
[29] G. Zhang, S. Zhang, W. Zhang, Z. Shen, and L. Wang, "Joint service caching, computation offloading and resource allocation in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 20, no. 8, pp. 5288–5300, Aug. 2021.
[30] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative UAV resource allocation and task offloading in hierarchical aerial computing systems: A MAPPO based approach," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10497–10509, Jun. 2023.
[31] H. Huang, C. Hu, J. Zhu, M. Wu, and R. Malekian, "Stochastic task scheduling in UAV-based intelligent on-demand meal delivery system," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 13040–13054, Aug. 2022.
[32] H. Zhou, Z. Wang, H. Zheng, S. He, and M. Dong, "Cost minimization-oriented computation offloading and service caching in mobile cloud-edge computing: An A3C-based approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 3, pp. 1326–1338, May/Jun. 2023.
[33] F. Chai, Q. Zhang, H. Yao, X. Xin, R. Gao, and M. Guizani, "Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite IOT," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7783–7795, Jun. 2023.
[34] Z. Sun, G. Sun, L. He, F. Mei, S. Liang, and Y. Liu, "A two time-scale joint optimization approach for UAV-assisted MEC," in *Proc. IEEE INFOCOM or IEEE Conf. Comput. Commun.*, 2024, pp. 91–100.
[35] L. Wang and G. Zhang, "Joint service caching, resource allocation and computation offloading in three-tier cooperative mobile edge computing system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3343–3353, Nov./Dec. 2023.
[36] G. Sun, et al., "Task delay and energy consumption minimization for low-altitude MEC via evolutionary multi-objective deep reinforcement learning," 2025, *arXiv:2501.06410*.



Dipankar Ch Barman received the B.Tech. degree in information technology from the Central Institute of Technology, Assam, India, in 2020, and the M.Tech. degree in computer science engineering from Tezpur University, Assam, India, in 2022. He is currently working toward the Ph.D. degree in information technology with the Indian Institute of Information Technology, Allahabad, India.

His research interests include unmanned aerial vehicles (UAVs), Internet of Things (IoT), and mobile edge computing (MEC).



Sayantan Chakraborty received the M.Tech. degree from the Indian Institute of Information Technology, Allahabad, India.

He is currently a Software Engineer with Qualcomm India Private Limited, Hyderabad, Telangana, India. His research interests include unmanned aerial vehicles (UAVs) and Internet of Things (IoT).



Abhishek Hazra (Member, IEEE) received the bachelor's degree from NIT, Agartala, India, in 2014, the master's degree from NIT, Manipur, India, in 2018, and the Ph.D. degree in computer science and engineering from Indian Institute of Technology (ISM) Dhanbad, India, in 2022.

He is currently an Assistant Professor with Indian Institute of Information Technology, Sri City, India. He has also served as a Research Fellow with the Communications and Networks Laboratory, Department of Electrical and Computer Engineering, National University of Singapore. His research interests include the IoT, edge computing, and machine learning.



Nabajyoti Mazumdar (Senior Member, IEEE) received the M.Tech. and Ph.D. degrees in Computer Science & Engineering from the Indian Institute of Technology (ISM), Dhanbad, India , 2014 and 2018 respectively, both in computer science and engineering.

He is currently an Assistant Professor with the Department of Information Technology, Indian Institute of Information Technology, Allahabad, India. His research interests include wireless sensor networks, the Internet of Things (IoT), and soft computing techniques.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:34:27 UTC from IEEE Xplore. Restrictions apply.